

06 · Explicación profunda del código

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 06-deep-code-explanation.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Cómo leer este documento
2. ``src/main.rs`` — el despachador
3. ``src/services/inspector.rs`` — el orquestador
4. ``src/services/macros.rs`` — el adaptador
5. ``src/services/launchd.rs`` — persistencia
6. ``src/services/security.rs`` — controles nativos
7. ``src/services/tcc.rs`` — privacidad
8. ``src/services/network.rs`` — conexiones
9. ``src/services/netscan.rs`` — vecinos de red
10. ``src/services/temp_scan.rs`` — almacenamiento
11. ``src/services/rules.rs`` — decisión
12. ``src/services/anomaly.rs`` — heurísticas
13. ``src/services/baseline.rs`` — el motor de cambios
14. ``src/services/persistence.rs`` — SQLite
15. ``src/services/resilience.rs`` — salud del agente
16. ``src/services/report.rs`` — el reporte forense
17. ``src/services/ai.rs`` — el adaptador opcional
18. ``src/app.rs`` — la interfaz
19. ``src/cli.rs`` — la consola
20. Casos límite recogidos en el código

Cómo funciona el código por dentro: qué hace cada módulo relevante, en qué orden, qué decide en cada bifurcación y qué pasa en los casos límite. Los fragmentos de código son mínimos y siempre citan archivo y símbolo.

1. Cómo leer este documento

Cada módulo se explica con la misma estructura: objetivo, entrada y salida, flujo interno, decisiones, casos límite y riesgos al modificarlo. Las funciones triviales (accesores, conversiones) se agrupan; las que concentran decisiones se explican bloque a bloque.

Orden recomendado de lectura: `main.rs` → `inspector.rs` → un recolector (`launchd.rs`) → `baseline.rs` → `anomaly.rs` → `rules.rs` → `persistence.rs`.

2. `src/main.rs` — el despachador

Objetivo. Elegir modo de ejecución sin inicializar nada innecesario.

Flujo, línea a línea de la parte que decide:

```
let args: Vec<String> = std::env::args().collect();
if args.len() > 1 && args[1] != "--gui" {
    std::process::exit(cli::run(&args[1..]));
}
```

1. Se recogen todos los argumentos, incluido `argv[0]`.
2. Si hay al menos un argumento **y no es `--gui`**, se entra en modo CLI. La condición está escrita en positivo sobre el primer argumento, así que `rootcause --json` (sin comando) también entra al CLI y acaba en la rama «comando desconocido» con código `1`.
3. `std::process::exit` propaga el código de retorno del CLI. No hay limpieza posterior, pero tampoco hace falta: el `Drop` de `InspectorService` ya se ejecutó al salir de `cli::run`, que es quien lo posee.

Después, dos ramas condicionales por *feature*:

- Con `gui`: `launch_gui()`; si falla, imprime el error en `stderr` y sale con `1`.
- Sin `gui`: llama a `cli::run(&["--help"])`, para que un binario CLI-only ejecutado sin argumentos no se quede en silencio.

Caso límite. `rootcause --gui` en la edición CLI-only: la condición del `if` lo excluye del CLI, y como no existe `launch_gui`, cae en el bloque `#[cfg(not(feature = "gui"))]` y muestra la ayuda. Comportamiento razonable, aunque no documentado explícitamente.

`rootcause_icon`. Genera un búfer RGBA de 64×64 dibujando dos anillos concéntricos y un punto central en el azul de marca. Se calcula la distancia euclídea de cada píxel al centro y se compara con tres radios. Está hecho a mano **para no depender de un decodificador de PNG ni de un recurso en disco**: el icono viaja dentro del binario como código.

3. `src/services/inspector.rs` — el orquestador

Es el archivo que hay que entender para entender el producto.

3.1 `InspectorService::new`

Objetivo. Dejar listo todo lo que sobrevive entre capturas.

Orden de operaciones y por qué:

1. `System::new_all()` + `refresh_all()` — primera lectura de `sysinfo`. Se hace ya para que la primera captura no pague el coste de inicialización.
2. `Networks::new_with_refreshed_list()` — lista de interfaces.

3. Se construye el conjunto de **trece nombres protegidos**. Es una lista literal, no configurable: matar `windowserver` o `launchd` deja el equipo inutilizable.
4. `PersistenceStore::new(meta::APP_DIR)` — crea la carpeta de datos y el esquema. Es el primer punto que puede fallar: si no hay carpeta de datos accesible, el servicio no arranca y quien llama lo explica.
5. `ConfigManager::load_or_default` — devuelve la configuración y **una advertencia opcional**. La advertencia se guarda en `config_warning` y se convierte en alerta en cada captura, hasta que el usuario arregle el JSON.
6. `ResilienceMonitor::new` — lee el estado de la sesión anterior y decide si hubo cierre abrupto.
7. Se vuelcan los `startup_audits()` del monitor a la tabla de auditoría.

Caso límite. Si el registro de auditoría falla, se ignora (`let _ = ...`): no tener auditoría de arranque no justifica impedir el arranque.

3.2 `collect_snapshot` — la función central

Es larga a propósito: es la secuencia completa de una captura. Bloque a bloque:

Bloque 1 — latido. `resilience_monitor.heartbeat()` primero, para que un cierre abrupto anterior se refleje en esta captura y no en la siguiente.

Bloque 2 — refresco de `sysinfo`. `system.refresh_all()` y `networks.refresh()`.

Bloque 3 — procesos. `macos::process_details()` hace **una sola** llamada a `ps` para todos los PID (llamarlo por proceso sería absurdamente caro), y `collect_processes` recorre `sysinfo` cruzando ambos.

Bloque 4 — firmas. Si `verify_signatures` está activo:

- `apply_signatures` elige a quién verificar. El criterio importa tanto como la verificación: entran los procesos **fuera de las rutas de confianza** o con severidad $\geq$ `Warning`, se ordenan por severidad y puntaje, y se toman los primeros `signature_budget` (12 por defecto).
- Cada ruta se resuelve una vez y se cachea en `signature_cache: codesign` cuesta un proceso y un binario no cambia de firma mientras corre.
- `reclassify_with_signatures` **vuelve a clasificar** solo los procesos con firma resuelta. Sin este paso, la firma no entraría en el puntaje, porque la clasificación inicial se hizo con `signature = None`.

Bloque 5 — orden. Los procesos se ordenan por severidad, luego puntaje, luego CPU. Ese orden determina cuál es el «proceso dominante» del historial y qué aparece primero en la tabla.

Bloque 6 — conexiones. Se construye un mapa `pid → ruta` y se pasa a `network::parse_lsof_field_output`, para que cada conexión pueda mostrar la ruta del binario sin volver a consultar el sistema.

Bloque 7 — superficies. Cachés, XProtect, controles de seguridad, TCC, persistencia y red. Cada una devuelve además su lista de cambios contra baseline mediante las funciones `detect_*_changes`.

Bloque 8 — resumen. Se rellena `SystemOverview` con CPU global, memoria, deltas de red y E/S y total de cachés. `primary_severity` empieza en `Healthy` con el texto «Sin señales fuertes en esta muestra»; lo eleva `build_alerts` si procede.

Bloque 9 — anomalías. `anomaly_tracker.analyze(...)` produce las heurísticas de comportamiento. Después se **añaden** los eventos de cambio de las cuatro superficies y se reordena todo junto:

«Los cambios contra baseline se añaden y todo se reordena junto: así un cambio de alta severidad no queda fuera del recorte de alertas por haberse detectado en otra fase.»

Ese comentario del código explica una decisión sutil: si se reordenara solo dentro de cada grupo, un cambio crítico de Gatekeeper podría quedar por debajo de una anomalía media simplemente por el orden de las fases.

Bloque 10 — alertas y veredicto. `rules::build_alerts` recibe todas las superficies y `&mut overview`, y devuelve las alertas ya recortadas a `max_alerts`.

Bloque 11 — advertencia de configuración. Si `config_warning` existe, se añade como alerta con la ruta del archivo y la pista de ejecutar `rootcause config init`.

Bloque 12 — salud del agente. `apply_agent_health` puede **eleva el veredicto global** a `Warning` e insertar una alerta en la posición 0 si el agente está degradado o recuperado. Al final trunca las alertas otra vez, porque acaba de insertar una.

Bloque 13 — incidente. `rules::derive_incident` devuelve `Option`: no toda captura genera incidente. Si lo hay, se persiste con `persist_incident`, que descarta duplicados inmediatos comparando la huella con la del último.

Bloque 14 — historial. `persist_snapshot`. Si falla, **no se pierde la captura**: se añade una alerta de advertencia («La app sigue funcionando; solo se pierde este punto del historial») y se devuelve la captura igualmente.

3.3 collect_processes y el problema de los deltas

`sysinfo` devuelve el total acumulado de E/S desde que arrancó el proceso, no lo que ha escrito en el intervalo. Restar contra cero haría pasar por «escritura del intervalo» toda la vida del proceso, lo que dispararía la heurística `aggressive-write` en la primera captura para cualquier proceso longevo.

La solución es el campo `seeded` de `ProcessIoBaseline`:

```
let (read_delta, write_delta) = if baseline.seeded {
  (total_read.saturating_sub(base.read), total_written.saturating_sub(base.write))
} else {
  baseline.seeded = true;
  (0, 0)
};
```

La primera muestra de cada PID **siembra** el contador y reporta delta 0. `saturating_sub` protege del caso en que el contador del sistema retroceda.

Al final, `process_baselines.retain(|pid, _| active_pids.contains(pid))` purga los PID muertos, para que el mapa no crezca sin límite en una sesión larga.

3.4 Las cuatro funciones detect_*_changes

Todas siguen el mismo patrón de tres pasos y merece la pena verlo una vez:

- 1. Convertir la superficie en `Vec<WatchedItem>` (`security::control_watch_items`, `tcc::permission_watch_items`, `netscan::device_watch_items`).
- 2. `baseline::diff_surface` las marca y devuelve si había baseline previa.
- 3. Propagar el `change_status` de vuelta a los modelos ricos, mediante un `HashMap` de clave → estado, y generar un `AnomalyEvent` por cada cambio.

Dos guardas importantes:

- Si **no había baseline previa** (`had_baseline == false`), no se genera ningún evento: es la primera foto y se siembra en silencio.
- Si la superficie está desactivada en la configuración (`watch_persistence`, `watch_security_controls`, `watch_tcc`, `watch_network_devices`), tampoco. Pero el `change_status` **sí se propaga**: la vista sigue mostrando la marca aunque no se genere alerta.

`detect_tcc_changes` añade una guarda propia al principio: si `overview.readable` es falso, devuelve vacío sin tocar la baseline. Sin ella, un arranque sin Acceso total al disco borraría la baseline de permisos y, al recuperar el permiso, reportaría **todos** los permisos como nuevos.

3.5 annotate_network_changes y los dispositivos ausentes

Además del diff normal, esta función reconstruye los dispositivos que estaban en la baseline y ya no responden (`device_from_watch_item`) y los añade a la lista, para que la vista muestre lo que desapareció. Después recalcula `total_devices` (excluyendo el propio equipo) y `new_devices`.

3.6 Acciones y su política

Acción	Guarda previa
<code>terminate_process</code>	<code>manual_actions</code> <code>enabled</code> → <code>pid != own_pid</code> → proceso existe → <code>can_terminate_process</code>

<code>clean_caches</code>	<code>dry_run</code> explícito; solo audita si borró de verdad
<code>suggest_block_ip</code>	Extrae la IP; nunca ejecuta <code>pfctl</code>
<code>accept_tcc_baseline</code>	Falla con mensaje claro si TCC no es legible
<code>login_items</code>	Solo se llama desde una acción explícita del usuario

`suggest_block_ip` merece una lectura completa porque es la decisión de producto más clara del archivo: construye el comando `pfctl` exacto, lo audita, y lo devuelve como texto. Aplicarlo requeriría root y modificar el firewall global del equipo; el código explica que eso debe ser una decisión consciente del usuario.

3.7 Drop y el cierre limpio

```
impl Drop for InspectorService {
    fn drop(&mut self) {
        if let Ok(record) = self.resilience_monitor.shutdown() {
            let _ = self.store.record_audit(&record);
        }
    }
}
```

Gracias a esto, un `rootcause status` que termina normalmente deja constancia de cierre limpio, y el siguiente arranque no lo interpreta como una caída. Si el proceso muere por `SIGKILL` o por un pánico abortando, `Drop` no se ejecuta y **eso es exactamente lo que el monitor de resiliencia quiere detectar**.

4. `src/services/macros.rs` — el adaptador

4.1 Dos formas de ejecutar

Función	Falla si el código ≠ 0	Une <code>stderr</code>	Para qué
<code>run_capture</code>	Sí	No	La mayoría: <code>ps</code> , <code>arp</code> , <code>sw_vers</code> , <code>lsof</code>
<code>run_combined</code>	No	Sí	<code>codesign</code> (escribe en <code>stderr</code>) y <code>spctl</code> (sale ≠ 0 en estados normales)

Esta distinción no es cosmética: usar `run_capture` con `codesign` devolvería error para **todos** los binarios, y usarla con `spctl --status` fallaría cuando Gatekeeper está desactivado, que es justo el caso que interesa detectar.

4.2 `process_details` — un `ps` para todos

Ejecuta `ps -axo pid=,user=,command=` y parsea con dos `split_once(char::is_whitespace)` sucesivos. El truco está en el segundo: la línea de comandos contiene espacios, así que se parte solo dos veces y el resto se queda entero.

Caso límite documentado: sin root, `ps` lista los procesos de todos los usuarios pero la línea de comandos de procesos ajenos puede aparecer recortada. Se acepta y se documenta en vez de pedir privilegios.

4.3 `classify_codesign_output` — orden de comprobaciones

El orden de los `if` es la lógica:

- `code object is not signed at all` → `Unsigned`. Va primero porque es inequívoco.
- Autoridades de Apple (`Software Signing`, `Apple Code Signing Certification Authority`, `Apple Root CA`) → `Apple`.

3. Autoridades de desarrollador (Developer ID Application, Apple Mac OS Application Signing, 3rd Party Mac Developer Application) → DeveloperId.
4. signature=adhoc o flags=0x2(adhoc) → AdHoc.
5. signature=none → Unsigned.
6. Cualquier otra cosa → Unknown.

La última línea es la decisión de seguridad: **lo que no se entiende no se asume de confianza**. Unknown tiene severidad Warning, no Healthy.

4.4 lsof_connections y el modo campo

```
const ARGS: &[&str] = &["-i", "-n", "-P", "-FpCLftPnT"];
run_capture("/usr/sbin/lsof", ARGS).or_else(|_| run_capture("/usr/bin/lsof", ARGS))
```

- -i sockets de red, -n sin resolver nombres, -P sin traducir puertos.
- -F... salida por campos: una línea por dato, prefijada por su letra.
- El or_else prueba la segunda ruta posible del binario.

El formato tabular de lsof se rompe con nombres como Google Chrome; el modo campo es feo de leer y trivial de parsear sin ambigüedad.

4.5 discovery_sweep — ruido deliberado

254 ping -c 1 -t 1 -q en serie. Es lento (hasta ~4 minutos en el peor caso) y ruidoso, y por eso **solo se ejecuta bajo petición explícita**. La alternativa —sockets raw o un escáner dedicado— exigiría privilegios o una dependencia nueva.

4.6 security_log_events — coste asumido

log show --last Nm --style compact --predicate ... acotado a seis procesos (syspolicyd, XProtect, XprotectService, tcdd, sudo, amfid). Tarda segundos, así que **nunca forma parte de la captura periódica**: se invoca desde la sección Historial o desde rootcause events.

El parseo salta la primera línea (cabecera), separa la marca de tiempo del resto y busca ":" para partir proveedor y mensaje. Si no encuentra el separador, usa "log" como proveedor: prefiere un dato imperfecto a descartar la línea.

5. src/services/launchd.rs — persistencia

5.1 scan_persistence

Recorre LAUNCH_DIRS saltando las carpetas de Apple salvo que include_apple sea cierto. Para cada archivo con extensión .plist llama a parse_launch_plist. Después añade las tareas de cron y ordena por severidad descendente y nombre ascendente.

Por qué se omiten las carpetas de Apple por defecto: son cientos de entradas inmutables protegidas por SIP; incluirlas llenaría la baseline de ruido sin aportar señal.

5.2 parse_launch_plist — el detalle de Program vs ProgramArguments

```
if let Some(program) = dict.get("Program")... { argv.push(program) }
if let Some(array) = dict.get("ProgramArguments")... {
    if argv.is_empty() { argv = args }
    else if args.len() > 1 { argv.extend(args.into_iter().skip(1)); }
}
```

launchd acepta las dos claves. Cuando conviven, Program manda sobre argv[0] y los elementos de ProgramArguments a partir del segundo son argumentos reales. Reproducir esa semántica importa: el comando resultante es lo que se compara contra la baseline, y una diferencia espuria generaría un falso «MODIFICADA».

Otros detalles:

- `KeepAlive` puede ser booleano o un diccionario de condiciones. El código usa `value.as_boolean().unwrap_or(true)`: cualquier diccionario significa «relánzame», que es lo que interesa marcar.
- `StartInterval` se filtra a valores positivos.
- La firma solo se calcula si `verify_signatures` está activo y el binario existe.
- Si el plist no se puede leer, devuelve `None`: un archivo corrupto no aborta el escaneo.

5.3 `classify_entry` — las siete señales

Ya tabuladas en [05 · Referencia técnica](#). Lo interesante es el diseño:

- El puntaje **parte del ámbito** (`base_risk`), así que el mismo comando pesa más en un LaunchDaemon de root que en un LaunchAgent de usuario.
- Las señales son **acumulativas y ninguna decide sola**. Un binario en `/tmp` (30) más ámbito daemon (26) ya son 56 → alto; si además está sin firmar (30) llega a 86 → crítico.
- La señal 3 (imitar `com.apple.*` fuera de las carpetas del sistema) es la de mayor peso individual (+35) porque es una técnica de camuflaje sin uso legítimo conocido.
- Si no hay ninguna señal y la nota está vacía, escribe «Sin señales anómalas en esta entrada»: la ausencia de hallazgo también se declara.

5.4 `login_items` y el permiso de Automatización

Ejecuta `osascript` contra System Events. La primera llamada dispara un diálogo de permiso de macOS, y por eso está fuera del escaneo automático. El comentario del código lo justifica: una herramienta de seguridad no debería provocar diálogos de permisos que el usuario no pidió.

6. `src/services/security.rs` — controles nativos

6.1 El patrón común de los seis controles

Cada función privada hace lo mismo: ejecuta un comando, pasa la salida a minúsculas, busca dos cadenas (una para «activo» y otra para «inactivo») y calcula:

- `enabled` — si la cadena de activo está presente.
- `known` — si alguna de las dos lo está.
- `severity` — vía `severity_for(enabled, known, cuando_apagado)`.

`severity_for` codifica la regla más importante del módulo:

```
match (known, enabled) {
  (true, true) => Severity::Healthy,
  (true, false) => when_disabled,
  (false, _)   => Severity::Warning,
}
```

«No lo sé» nunca se pinta de verde. Si el comando no respondió o su salida cambió en una versión nueva de macOS, el control aparece en amarillo como «Desconocido».

6.2 Las tres excepciones al patrón

1. **Firewall de aplicaciones** tiene respaldo: si `socketfilterfw` no da un estado reconocible, lee `/Library/Preferences/com.apple.alf` con `defaults`. Acepta `state = 1` y `state = 2` como activo (bloqueo selectivo y bloqueo total).
2. **Modo encubierto** nunca sube de verde. Viene apagado de fábrica; alarmar por el valor por defecto de Apple sería ruido.
3. **Acceso remoto SSH** invierte la semántica: aquí «el servicio está activo» es lo inseguro, así que `enabled` guarda la negación y la severidad se calcula aparte. Se deduce de `launchctl list` porque `systemsetup -getremotelogin` exige root.

6.3 scan_xprotect — deduplicación y antigüedad

Recorre las cuatro rutas conocidas, deduplica por `componente::versión` (las rutas nueva y heredada pueden apuntar a la misma versión) y calcula la antigüedad como `(ahora - fecha_de_modificación_del_bundle).num_days().max(0)`.

Caso límite explícito: si no se pudo leer ninguna definición, `freshest_age_days` sería `i64::MAX`, así que se normaliza a `-1` y la severidad queda en `Warning` con el titular «No se pudieron leer las definiciones».

La limitación se declara en la propia salida: *la antigüedad se mide por la fecha del bundle en disco, no por la fecha de publicación de Apple.*

7. src/services/tcc.rs — privacidad

7.1 El problema del esquema

`TCC.db` cambió de esquema: hasta High Sierra la columna era `allowed` (booleano); desde Mojave es `auth_value` (entero). El código **no asume una versión**: pregunta.

```
let columns = table_columns(&connection, "access")?;
let has_auth_value = columns.iter().any(|c| c == "auth_value");
```

Y construye la consulta con el nombre correcto. Lo mismo con `last_modified`, que en esquemas antiguos no existe: si falta, se consulta el literal `0` como columna, que `format_epoch` traduce a cadena vacía.

7.2 decode_decision

Valor	Esquema moderno	Esquema heredado
<code>0</code>	denegado (no permitido)	denegado
<code>1</code>	desconocido (no permitido)	permitido
<code>2</code>	permitido	permitido
<code>3</code>	limitado (sí cuenta como permitido)	permitido

Que `1` sea «desconocido» y no cuente como concedido es deliberado: en TCC ese valor significa que el usuario aún no ha respondido.

7.3 Severidad y ruido

`severity_for` devuelve `Healthy` para cualquier permiso **no concedido** —un permiso denegado que sigue denegado no aporta nada— y `Warning` para los sensibles concedidos. Ninguno llega a `Critical`: muchas aplicaciones legítimas necesitan estos permisos, y pintar de rojo a la app de videollamadas por tener el micrófono destruiría la señal.

`permission_watch_items` filtra dos veces (concedido y sensible) antes de construir la baseline. Vigilarlo todo llenaría la tabla de cientos de filas irrelevantes.

7.4 La ironía documentada

Leer `TCC.db` exige Acceso total al disco. El módulo lo asume y responde con honestidad: si la lectura falla, `readable = false`, `full_disk_access = false`, y las limitaciones incluyen la ruta exacta de Ajustes donde concederlo. `rules::build_alerts` convierte eso en una alerta explícita: **la ausencia de dato es un dato**.

8. src/services/network.rs — conexiones

8.1 La máquina de estados del parser

`lsOf -F` emite líneas `<letra><valor>`. El parser mantiene un `FieldState` y emite un registro cuando empieza uno nuevo:

Letra	Significado	Efecto en el parser
<code>p</code>	PID	Cierra el descriptor abierto y empieza un proceso nuevo
<code>c</code>	Comando	Guarda el nombre
<code>L</code>	Usuario	Guarda el usuario
<code>f</code>	Descriptor	Cierra el anterior y abre uno nuevo ; limpia protocolo, familia, nombre y estado
<code>t</code>	Familia (IPv4/IPv6)	Respaldo de protocolo
<code>P</code>	Protocolo (TCP/UDP)	—
<code>n</code>	Nombre (<code>local->remoto</code>)	—
<code>T</code>	Estado TCP (<code>ST=ESTABLISHED</code>)	Solo se guarda el prefijo <code>ST=</code>

Al terminar el bucle hay que emitir el último descriptor pendiente: es el fallo clásico de este tipo de parsers y el código lo cubre con un `if has_descriptor` final.

Detalle sutil: los campos `p`, `c` y `L` pertenecen al proceso y los demás al descriptor. Por eso `'p'` limpia comando y usuario, pero `'f'` no: un proceso con cinco sockets emite `c` y `L` una vez y `f` cinco veces.

8.2 `classify_connection`

Tres ramas, en orden:

- 1. **Socket a la escucha.** Si la dirección local empieza por `*:`, `0.0.0.0:` o `:::`, está expuesto a toda la red → `Warning`. Si escucha solo en la interfaz local → `Healthy`. Es la señal más accionable del módulo, porque distingue «tengo un servicio corriendo» de «tengo un servicio alcanzable desde fuera».
- 2. **Destino público** → `Warning` con la IP en el motivo.
- 3. **Resto** → `Healthy`.

8.3 `is_public_ip` — qué no es Internet

Además de los rangos obvios, excluye explícitamente el CGNAT `100.64.0.0/10` (`octets[0] == 100 && (64..128).contains(&octets[1])`). Sin esa exclusión, cualquier equipo detrás del NAT del operador vería sus conexiones internas marcadas como públicas.

Para IPv6 comprueba `fc00::/7` (unique local) y `fe80::/10` (link-local) con máscaras de bits sobre el primer segmento.

Si la cadena no parsea como IP, devuelve `false`: `lsOf` puede devolver un nombre de host si no se usó `-n`, y un nombre no es una IP pública reconocible.

9. `src/services/netscan.rs` — vecinos de red

9.1 `parse_arp_table`

Formato de entrada: `? (192.168.1.1) at 0:11:22:33:44:55 on en0 ifscope [ethernet]`.

Extracción por posición relativa, sin expresiones regulares:

- 1. IP entre paréntesis: `split_once('(') → split_once('')`.
- 2. MAC después de `" at "`, primer token.

3. Interfaz después de " on ", primer token.

Descarta entradas (`incomplete`), tokens sin `:` y la MAC de difusión `ff:ff:ff:ff:ff:ff`.

9.2 `normalize_mac` — el bug que evita

macOS imprime los octetos sin cero a la izquierda: `0:11:22:33:44:55`. Si se guardara así en la baseline, la comparación con una lectura posterior que lo formateara distinto marcaría un cambio inexistente. `normalize_mac` rellena cada octeto a dos dígitos y pasa a minúsculas, de modo que la clave es estable.

9.3 `classify_device` — el caso crítico

```
PersistenceChange::Added if device.is_gateway => Critical,  
PersistenceChange::Added                               => Warning,  
PersistenceChange::Modified                             => Warning,  
PersistenceChange::Removed                             => Healthy,
```

La primera rama es la razón de ser del módulo: la clave de la baseline es la MAC, así que una puerta de enlace «nueva» significa que **el router responde con otra MAC**. En una red doméstica eso es, casi siempre, una suplantación ARP. El evento correspondiente lleva puntaje 92 y la acción recomendada es «verifica físicamente el router antes de seguir usando la red».

Un dispositivo que desaparece baja a `Healthy`: apagar un portátil es normal.

9.4 El propio equipo

El Mac casi nunca aparece en su propia tabla ARP, así que `scan` lo añade a mano con `is_self = true`. `device_watch_items` lo excluye de la baseline —vigilarse a sí mismo no aporta— y `total_devices` no lo cuenta.

10. `src/services/temp_scan.rs` — almacenamiento

10.1 Medición acotada

`measure_directory` recorre con `WalkDir` sin seguir enlaces simbólicos (seguirlos podría salir del árbol o entrar en un bucle) y **corta a las 40 000 entradas**, devolviendo un tercer valor `truncated` que la vista convierte en una limitación explícita: «Medición aproximada (tope de 40 000 entradas) en: ...».

Un monitor que tarda un minuto en refrescar deja de usarse; esta es la concesión consciente de precisión a favor de la utilidad.

10.2 `clean_user_caches` — tres salvaguardas y media

1. **Solo `~/Library/Caches`**. Ni `/Library/Caches`, ni la papelera, ni temporales del sistema, aunque los mida.
2. **Solo lo no modificado en `min_age_hours`** (24 en todas las llamadas actuales).
3. **Salta lo que esté en uso**. `PermissionDenied` y `ErrorKind::Other` (que en macOS cubre `ResourceBusy`) incrementan `skipped_in_use` en vez de forzar el borrado.
4. **`dry_run` calcula exactamente lo mismo sin borrar**, y es el valor por defecto del CLI: borrar requiere pedirlo dos veces (`clean-caches` y luego `--yes`). En la GUI, el equivalente es el campo `clean_armed`.

REQUIERE VALIDACIÓN: el mapeo de `ResourceBusy` a `ErrorKind::Other` depende de la versión de Rust; en toolchains recientes existe `ErrorKind::ResourceBusy` como variante propia y podría dejar de coincidir con `Other`. En ese caso el borrado no fallaría, pero el archivo se contaría en `error_count` en vez de en `skipped_in_use`.

11. `src/services/rules.rs` — decisión

11.1 `classify_process`

Función pura con siete bloques de suma, todos con `saturating_add` para que el `u8` no desborde. Devuelve (`severidad`, `puntaje`, `motivos`, `categoría`).

Detalles que importan:

- Los umbrales llegan por parámetro (`ProcessThresholds`), así que la función no lee configuración global y se puede probar con valores fijos.
- La categoría se calcula antes de la última suma porque «Instalador / actualizador» añade 10 puntos: un actualizador escribiendo mucho es más interesante que un editor de vídeo haciendo lo mismo.
- Si no hay ningún motivo, escribe «Sin presión relevante en esta muestra». La lista de motivos nunca queda vacía, lo que simplifica la interfaz.

El test `una_cpu_alta_por_si_sola_no_es_critica` fija la intención con un comentario explícito: *compile no debería pintarse de rojo*.

11.2 `build_alerts` — el orden es la política

Ocho fuentes, en este orden: anomalías correlacionadas → controles apagados → XProtect desactualizado → persistencia de riesgo alto (máx. 3) → procesos dominantes (máx. 3) → puerto expuesto (el primero) → TCC ilegible → cachés voluminosas (la primera).

Después ordena por severidad descendente (`sort_by_key(|a| Reverse(a.severity))`) y trunca a `max_alerts`. El orden de inserción actúa como **desempate**: entre dos alertas de la misma severidad, sobrevive la más específica.

El veredicto global se fija con la primera alerta, y **solo si supera** la severidad que ya traía `overview`. Así, un veredicto ya elevado por otra vía no se degrada.

11.3 `derive_incident` y la huella

Un incidente se genera si hay una anomalía de severidad $\geq$ `Medium` o una alerta crítica. La severidad final es el máximo entre la de la anomalía y `Critical` si había alerta crítica.

La huella es la clave de la deduplicación:

```
let fingerprint = format!("{kind}|{proceso}|{title}");
```

`persist_incident` compara esa huella con la del último incidente guardado y descarta el duplicado. Sin esto, una condición persistente generaría un incidente **por captura**, es decir, uno cada cinco segundos.

El test `la_huella_agrupa_incidentes_equivalentes` fija las dos mitades de la regla: la huella debe coincidir y el `incident_id` debe diferir, porque cada incidente conserva su instante.

12. `src/services/anomaly.rs` — heurísticas

12.1 Estado por proceso

```
struct ProcessHistory {
    cpu_streak: u8,
    write_streak: u8,
    memory_baseline_mb: f32,
    memory_growth_streak: u8,
    last_seen: Option<DateTime<Utc>>,
}
```

Los contadores de racha se **incrementan o se ponen a cero** en cada muestra, y el evento se dispara con `==` al valor configurado, no con `>=`. Esa igualdad estricta es lo que evita que la misma condición genere un evento en cada captura mientras dura: se emite una vez al cruzar el umbral.

12.2 Crecimiento de memoria

Es la única heurística con **reinicio de línea base**: al disparar, actualiza `memory_baseline_mb` al valor actual y pone la racha a cero, de modo que un proceso que crece sostenidamente reporta un evento por escalón y no uno por muestra. Si la memoria **baja**, la línea base se reajusta hacia abajo y la racha se reinicia.

12.3 La guarda `installed_normally`

```
let installed_normally = is_trusted_path(&lower_path_of(process), input.config)
    && !matches!(process.signature, Some(CodeSignature::Unsigned));
```

Las dos heurísticas de red (`unusual-outbound` y `local-scan`) solo se aplican a procesos que **no** cumplan esa condición. Un navegador en `/Applications` habla con decenas de destinos públicos por diseño; la señal está en que lo haga algo que vive fuera de esas rutas o que no está firmado.

Los tests `un_navegador_instalado_con_normalidad_no_dispara_trafico_inusual` y `un_binario_sin_firmar_en_applications_si_dispara_trafico_inusual` fijan las dos mitades.

12.4 `track_respawns` — por qué está fuera del bucle

Detecta procesos que mueren y renacen con PID nuevo. Dos decisiones:

1. **Se resuelve fuera del bucle principal.** Dentro, un nombre con varias instancias vivas (`Google Chrome Helper`, `mdworker_shared`) parecería reaparecer en cada iteración y generaría un falso positivo en la primera captura.
2. **Solo se vigilan los nombres con una única instancia viva.** Si un nombre pasa a tener varias, se borra su rastro para que no arrastre estado.

La ventana temporal caduca (`respawn_window_secs`, 180 s): reinicios repartidos en horas no deben acumularse hasta disparar un falso positivo.

12.5 `is_trusted` — el atajo de Apple

```
matches!(process.signature, Some(CodeSignature::Apple))
    && is_trusted_path(&process.exe_path.to_ascii_lowercase(), config)
```

Un binario firmado por Apple en una ruta del sistema se salta todas las heurísticas. El comentario del código lo justifica sin adornos: *si eso está comprometido, el problema es de otro orden.*

12.6 `persistence_change_event` y la escalada

Un cambio en la baseline de persistencia es `High` (nueva/modificada) o `Medium` (eliminada). Pero si la entrada **además** es sospechosa por sí misma (`entry.severity >= High`), escala a `Critical`: dos señales independientes apuntando al mismo sitio.

13. `src/services/baseline.rs` — el motor de cambios

13.1 `diff_surface`, paso a paso

```
let Ok(baseline) = store.load_baseline(surface_id) else { return false };
if baseline.is_empty() {
    let _ = store.replace_baseline(surface_id, items);
    return false;
}
```

1. Si la lectura falla, se devuelve `false` **sin marcar nada**: mejor no reportar que reportar mal.
2. Si la baseline está vacía, se siembra con el estado actual y se devuelve `false`. Es la primera foto.
3. Con baseline previa, cada ítem se compara por clave: ausente → `Added`; valor distinto → `Modified`; igual → `Unchanged`.
4. Los ítems de la baseline que no aparecen ahora se añaden como sintéticos `Removed`.

13.2 Por qué los cambios son pegajosos

`diff_surface` **no actualiza la baseline** cuando encuentra cambios. Solo la escriben `replace_baseline` (siembra inicial) y las acciones explícitas `accept_*_baseline`. Es la decisión de producto más importante del módulo, y el propio código la explica: *una alerta que se auto-silencia tras un reinicio es peor que no tenerla.*

13.3 La clave de persistencia

`persistence_entry_key` usa `kind■location■name` con el separador de unidad `\u{1f}` y **deliberadamente no incluye el comando**. Consecuencia: cambiar el comando de un plist existente se lee como *modificación*, no como una entrada eliminada más una nueva. El test `la_clave_ignora_el_comando` fija esa intención.

14. `src/services/persistence.rs` — SQLite

14.1 Una conexión por operación

Cada método abre su propia `Connection::open(&self.db_path)`. No hay pool ni conexión compartida. Para el volumen real (una escritura cada cinco segundos, lecturas puntuales) es suficiente, y evita por completo los problemas de compartir una conexión entre hilos.

INFERENCIA: con un intervalo de refresco muy bajo y un historial grande, esto podría notarse. No se ha medido.

14.2 `trim_table` y la inyección que no es

```
connection.execute(&format!(
    "DELETE FROM {table} WHERE id NOT IN (SELECT id FROM {table} ORDER BY id DESC LIMIT ?1)"
), params![keep as i64]);
```

El nombre de tabla se interpola porque SQLite no acepta parámetros para identificadores. El código lo justifica en un comentario: `table` nunca viene del usuario: son literales de este módulo. Las dos únicas llamadas pasan `"snapshots"` e `"incidents"`.

14.3 Las baselines y las entradas sintéticas

Tanto `replace_persistence_baseline` como `replace_baseline` **saltan los ítems marcados `Removed`** antes de insertar. Sin esa guarda, aceptar una baseline que contiene entradas sintéticas de «eliminada» las reintroduciría como si existieran, y volverían a reportarse como eliminadas en la captura siguiente: un bucle.

Ambas operan dentro de una transacción (`DELETE + INSERT`), de modo que un fallo a mitad no deja la baseline vacía.

14.4 `persist_incident` y el duplicado inmediato

Compara la huella con la del **último** incidente, no con todas. Es una deduplicación de ventana 1: si la condición A se repite, se sustituye por B y vuelve A, se guardan tres incidentes. Es intencional —A volvió a ocurrir después de otra cosa— aunque el código no lo declare: **INFERENCIA**.

15. `src/services/resilience.rs` — salud del agente

15.1 Cómo se detecta un cierre abrupto

```
let unexpected = !previous.last_start_at.is_empty()
    && previous.last_clean_shutdown_at
        .as_deref()
        .map(|value| value < previous.last_start_at.as_str())
        .unwrap_or(true);
```

La comparación es **de cadenas**, no de fechas. Funciona porque ambas son RFC 3339 en UTC, formato en el que el orden lexicográfico coincide con el cronológico. Es un atajo válido mientras las dos marcas se generen igual (`Utc::now().to_rfc3339()`), y frágil si alguna vez se guardara una con desplazamiento horario: **INFERENCIA** sobre el riesgo, no un fallo actual.

15.2 Ventana de reinicios

Si hubo cierre abrupto, se comprueba si la ventana (`restart_window_secs`, 600 s) caducó. Si caducó, el contador se reinicia a 1 y la ventana empieza de nuevo; si no, se incrementa. Alcanzar `max_restarts_in_window` (3) pasa el estado a `Degraded`.

15.3 La huella de configuración, sin vender humo

```
format!("{}", metadata.len(), modified)
```

Tamaño y fecha de modificación. El código lo dice en su propia documentación: *no es un hash criptográfico a propósito — no defiende contra un atacante que quiera falsificarla, y decir lo contrario sería vender humo*. Sirve para notar que el archivo cambió entre sesiones, que es exactamente lo que promete.

15.4 Cómo llega al veredicto

`inspector::apply_agent_health` traduce el estado a la captura: `Recovered` y `Degraded` elevan `primary_severity` a `Warning` si no era ya mayor, e insertan una alerta en la posición 0. `Healthy` no añade nada, lo que el test `un_agente_sano_no_agrega_ruido` comprueba.

16. `src/services/report.rs` — el reporte forense

Construye una cadena Markdown de once secciones concatenando con `push_str`. No usa plantillas ni motor de renderizado: para un documento de estructura fija, un `String` es suficiente y no añade dependencias.

Dos detalles de robustez:

- `escape` reemplaza `|` por `\|` y los saltos de línea por espacios. Sin eso, un mensaje de error con una barra vertical rompería la tabla Markdown. El test `las_barras_no_rompen_las_tablas` lo fija.
- `nonempty` sustituye los campos vacíos por `-`, para que el documento no tenga celdas en blanco que parezcan un error de generación.

La sección 11 concatena las limitaciones de cachés, TCC y XProtect, y añade siempre la línea que define el producto: *RootCause es un sensor forense y de apoyo a la decisión: no elimina malware ni sustituye a un EDR*. El cierre declara que ningún dato salió del equipo, y hay un test que lo comprueba.

17. `src/services/ai.rs` — el adaptador opcional

17.1 Tres guardas antes de tocar la red

```
if !self.config.enabled { bail!("La integración IA está desactivada...") }
if self.config.endpoint.trim().is_empty() { bail!("Falta `ai.endpoint`...") }
let api_key = env::var(&self.config.api_key_env_var)...?;
```

En ese orden. Con la configuración por defecto, la primera guarda corta: **no se construye payload ni se lanza `curl`**. El test `la_ia_desactivada_falla_sin_tocar_la_red` lo fija.

17.2 Qué viaja exactamente

`build_payload` incluye título, tipo, resumen, hipótesis local y evidencia del incidente, más un prompt de sistema que pide respuesta en JSON y prohíbe afirmar infección sin evidencia. **No incluye** la captura, ni la lista de procesos, ni rutas del usuario, ni permisos TCC. El test `el_payload_solo_lleva_el_incidente_resumido` comprueba precisamente que la cadena `"TCC"` no aparece.

17.3 Por qué `curl` y por qué `stdin`

- `curl` viene con macOS. Añadir un cliente HTTP a las dependencias por una función opcional no compensa, y aumentaría la superficie de dependencias de un producto de seguridad.
- **El cuerpo va por `stdin`** (`--data @-`) para que no aparezca en la lista de procesos. La clave viaja en una cabecera, que también sería visible en `ps` si se pasara como argumento; aquí se pasa como argumento de `-H`, lo que **sí es visible**: se documenta como riesgo en [11 · Seguridad](#).

17.4 Parseo defensivo

`parse_response` navega `choices[0].message.content` con `and_then` encadenados y falla con un mensaje claro si falta cualquier eslabón. Después deserializa el contenido —que es JSON dentro de una cadena JSON— en `AiOutputShape`, cuyos campos opcionales llevan `#[serde(default)]`. Si el modelo devuelve menos campos de los pedidos, no rompe.

18. `src/app.rs` — la interfaz

18.1 El contrato con el hilo de trabajo

```
fn spawn_worker(ctx: Context) -> (Sender<Command>, Receiver<EngineEvent>)
```

El hilo **posee** el `InspectorService`. Si la construcción falla, envía `EngineEvent::Failed` y termina: la ventana sigue abierta y explica el problema en vez de cerrarse de golpe.

El bucle `while let Ok(command) = command_rx.recv()` bloquea hasta recibir un comando, ejecuta la acción, envía la respuesta y llama a `ctx.request_repaint()`. `Command::Shutdown` rompe el bucle.

El enum se llama `EngineEvent` y no `Response` por una razón práctica documentada en el código: `egui::Response` es el tipo de retorno de todos los widgets de este módulo.

18.2 El ciclo de un frame

`update()` hace siempre lo mismo, en este orden:

1. `drain_responses()` — vacía la cola sin bloquear (`try_recv`).
2. `apply_theme(ctx)` — resuelve el tema (con `system_prefs_dark()` si es `System`).
3. `maybe_auto_refresh()` — pide captura si pasó el intervalo y no hay trabajo en curso.
4. Atajos de teclado (F5, ■E, ■R).
5. Dibuja barra lateral, superior, de estado y la sección activa.
6. `request_repaint_after(120 ms | 900 ms)`.

Caso límite: `maybe_auto_refresh` fuerza un mínimo de 2 segundos (`refresh_interval_secs.max(2)`) aunque la configuración permita menos, para que un valor absurdo no convierta la app en un bucle de capturas.

18.3 Estado global de la paleta

```
static mut ACTIVE_DARK: bool = true;
fn pal() -> Palette { if unsafe { ACTIVE_DARK } { DARK } else { LIGHT } }
```

Es la única concesión a `unsafe` del proyecto. El comentario acota la garantía: *solo se lee y escribe desde el hilo de la interfaz*, que en `egui` es siempre el mismo. La alternativa —pasar la paleta por parámetro a las más de treinta funciones de dibujo— se descartó por ruido. Se anota como deuda técnica menor en [15 · Riesgos](#).

18.4 Guardado de configuración

`draw_config` acumula un booleano `changed` con `|=` en cada control y, al final, guarda si hubo cambios o si se pulsó el botón. Es decir: la configuración se guarda sola al mover un deslizador. Cómodo, pero implica una escritura de disco por interacción:

INFERENCIA sobre el impacto, no medido.

19. `src/cli.rs` — la consola

19.1 Parseo sin dependencias

Tres funciones de tres líneas cada una (`wants`, `flag_value`, `first_number`) resuelven todo el parseo. No hay validación de banderas desconocidas: `rootcause status --inventada` se ejecuta ignorando la bandera. Es una decisión implícita, no documentada: **INFERENCIA**.

`flag_value` no entra en pánico si la bandera es el último argumento (`args.get(index + 1)` devuelve `None`), y hay un test que lo fija.

19.2 truncate y los caracteres multibyte

```
if value.chars().count() <= max { return value.to_owned(); }
let mut out: String = value.chars().take(max.saturating_sub(1)).collect();
out.push('...');
```

Cuenta **caracteres**, no bytes. Cortar por bytes en una cadena con acentos —habitual en este producto, que está en español— provocaría un pánico por índice no alineado. El test `recorta_respetando_caracteres_multibyte` usa `"ñññññ"` para fijarlo.

19.3 El patrón de cada comando

```
let Ok(mut service) = service() else { return 1 };
let snapshot = match service.collect_snapshot() { Ok(s) => s, Err(e) => { eprintln!(...); return 1 } };
if wants(args, "--json") { return print_json(&snapshot); }
// ... impresión tabulada
```

Los comandos que solo necesitan una superficie (`security`, `tcc`, `xprotect`) llaman directamente al módulo correspondiente en vez de hacer una captura completa: es mucho más rápido. Aun así construyen el servicio, porque necesitan la configuración y la auditoría.

20. Casos límite recogidos en el código

Caso	Dónde se resuelve	Comportamiento
Primera vez que se ve un PID	<code>collect_processes</code>	Delta de E/S = 0
Primera baseline de una superficie	<code>diff_surface</code>	Se siembra en silencio
<code>TCC.db</code> ilegible	<code>tcc::scan + detect_tcc_changes</code>	<code>readable = false</code> , alerta, baseline intacta
Plist corrupto	<code>parse_launch_plist</code>	Se salta esa entrada
MAC con octetos sin cero	<code>normalize_mac</code>	Se normaliza antes de comparar
Nombre de proceso con espacios	<code>parse_lsof_field_output</code>	Modo campo, sin ambigüedad
Nombre multi-instancia	<code>track_respawns</code>	Se ignora para reapariciones
Definiciones de XProtect ilegibles	<code>scan_xprotect</code>	<code>freshest_age_days = -1</code> , severidad <code>Warning</code>
Caché enorme	<code>measure_directory</code>	Corte a 40 000 entradas y limitación declarada
Archivo de caché en uso	<code>clean_user_caches</code>	Se cuenta como saltado, no se fuerza
Configuración JSON inválida	<code>load_or_default</code>	Valores por defecto + alerta persistente
SQLite no escribible	<code>collect_snapshot</code>	Alerta de advertencia; la captura se devuelve igual

Proceso protegido	<code>can_terminate_ process</code>	Error explícito y auditado
Barra vertical en un texto del reporte	<code>report::escape</code>	Se escapa para no romper la tabla
Cadena con acentos en una tabla del CLI	<code>cli::truncate</code>	Corte por caracteres
IA sin configurar	<code>summarize_ incident</code>	Error claro sin tocar la red

Siguiente lectura recomendada: [07 · Base de datos](#) para el detalle de lo que se persiste, o [08 · Flujo de datos](#) para seguir un dato de punta a punta.